

```

typedef struct ssl3_record_st
{
    int type; /* type of record */
    unsigned int length; /* How many bytes available */
    unsigned int off; /* read/write offset into 'buf' */
    unsigned char *data; /* pointer to the record data */
    unsigned char *input; /* where the decode bytes are */
    unsigned char *comp; /* only used with decompression - malloc()ed */
    unsigned long epoch; /* epoch number, needed by DTLS1 */
    unsigned char seq_num[8]; /* sequence number, needed by DTLS1 */
} SSL3_RECORD;

```

```

1448 dtls1_process_heartbeat(SSL *s)
1449 {
1450     unsigned char *p = &s->s3->rrec.data[0], *pl;
1451     unsigned short hbtype;
1452     unsigned int payload;
1453     unsigned int padding = 16; /* Use minimum padding */
1454
1455     /* Read type and payload length first */
1456     hbtype = *p++;
1457     n2s(p, payload);
1458     pl = p;
1459
1460     ...
1465     if (hbtype == TLS1_HB_REQUEST)
1466     {
1467         unsigned char *buffer, *bp;
1468
1469         ...
1470         /* Allocate memory for the response, size is 1 byte
1471          * message type, plus 2 bytes payload, plus
1472          * payload, plus padding
1473          */
1474         buffer = OPENSSL_malloc(1 + 2 + payload + padding);
1475         bp = buffer;
1476
1477         /* Enter response type, length and copy payload */
1478         *bp++ = TLS1_HB_RESPONSE;
1479         s2n(payload, bp);
1480         memcpy(bp, pl, payload);

```

```

/* Naive implementation of memcpy()
void *memcpy (void *dst, const void *src, size_t n)
{
    size_t i;
    for (i=0; i<n; i++)
        *(char *) dst++ = *(char *) src++;
    return dst;
}

```